Lecture Slip, CS 235
Nov 10, 2026, Tuesday
Searching Algorithms

Name:	
EmpID:	

1. **Binary search trace.** Binary search (the `lo <= hi` version from class) runs on `a = {3, 8, 12, 19, 25, 31, 44, 52, 60}` (indices 0 to 8) with target **20**. Here `mid = lo + (hi - lo) / 2`. Rows 1 and 2 are done for you. Complete the table. Use as many rows as you need.

Step	`lo`	`hi`	`mid`	`a[mid]`	**Next range**
1	0	8	4	25	$25 > 20$, so `hi = mid - 1 = 3`
2	0	3	1	8	$8 < 20$, so `lo = mid + 1 = 2`
3					
4					
5					

   What does the search return? How many times was `a[mid]` compared with the target? (Do not count the `lo <= hi` test of the loop.)

2. **Growth and data structures.**

   (a) In the worst case, how many comparisons does binary search need for $n = 1000$ sorted numbers? For $n = 2000$?

   (b) A programmer says: "Binary search halves the range each time, so on a `std::list` (a linked list) it is also $O(\log n)$." What is wrong with that claim?

3. **A one-line bug.** In the `lo <= hi` version, a student changes `hi = mid - 1;` to `hi = mid;`.

   (a) Run the changed code on `a = {5, 7}` with target **3**. Give `lo`, `hi`, `mid` for the first two iterations.

   (b) What happens next? State the rule that was broken.

4. **Pace survey.** Please help me set the pace of the course. Circle one.

Too **slow** (I can go faster)	**About right** (normal pace)	Too **fast** (please slow down)

5. **Topics to clarify.** Check **all** topics you would like me to explain again next class.

- ☐ Linear search and its cost
- ☐ Binary search code (`lo`, `hi`, `mid`)
- ☐ Tracing a search by hand
- ☐ Why binary search is $O(\log n)$
- ☐ Arrays versus linked lists for searching
- ☐ The off-by-one bugs (`mid + 1`, `mid - 1`)
- ☐ The code `tokenize`
- ☐ The code `toPostfix` (shunting-yard)
- ☐ The code `evalPostfix`
- ☐ Other: _____

6. Any other questions about today's class? If so, please provide details. Thank you.